GOSIM

GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM Paris 2026

Topic

Vulkan for Edge AI: Expanding the Hardware Frontier with llama.cpp

Ruben Ortlam

Senior Machine Learning Engineer, Red Hat

Vulkan for Edge AI

Expanding the Hardware Frontier with llama.cpp

Ruben Ortlam
Red Hat

CONTENTS

01

Edge AI Landscape

Why bother with local inference?

Local Inference

- Data stays on device
- No network latency
- Works offline
- No per-token cost
- Full control & customization

Internet

VS

Cloud API

- No local hardware needed
- Largest models available
- Data leaves your device
- Ongoing API costs

Edge AI Hardware

- What counts as edge?
- Any computer likely to be owned by regular people
  - Gaming PCs
  - Laptops
  - Smartphones
  - Home servers
- Amount of compute may be enough for some tasks
- New CPUs with integrated GPUs are well-suited for Mixture-of-Experts LLMs
  - Nvidia DGX Spark
  - AMD Ryzen AI Max+ 395
  - Apple M-chips

Compute Accelerator APIs

API	Supported Devices	Open Source?	Maturity	Feature Support
CUDA	Nvidia	No	High	High
ROCm	AMD	Yes	Medium	High
OpenCL	Many	Yes	High	Medium
SYCL	Many	Yes	Low	High
Vulkan	Many	Yes	High	High

...

Vulkan as Vendor-Neutral Compute API

Vendor	Vulkan Support Since	Float16 Support Since	Coopmat Support Since
Nvidia	Kepler (GTX 600)	Tesla P100 / Volta/Turing	Volta/Turing (RTX 2000)
AMD	GCN1 (HD 7000)	GCN5 (Vega)	RDNA3 (RX 7000)
Intel	Skylake	Ice Lake	Battlemage* (Arc B…)

One major exception: AMD CDNA accelerators have no Vulkan support.

* Technically Alchemist, but performance is poor

02

llama.cpp / GGML and Vulkan

The llama.cpp Vulkan Backend

- local inference engine with focus on compatibility
- compute-graph based, with device backend abstraction
- mature Vulkan backend
  - ~150 shaders, ~15,000 lines of shader code, ~15,000 lines of host code
  - FP16/BF16/int8 hardware acceleration support
  - Tensor/Matrix Core support through coopmat and coopmat2
  - Focus on Nvidia, AMD and Intel hardware
  - Competitive Performance
  - Small Binary Size

The llama.cpp Vulkan Backend

Applications & Tools
examples/ , tools/server , CLI interfaces

llama.cpp High-Level API
llama_model , llama_context , llama_generate() | src/llama.cpp , include/llama.h

Model Loading & Graph Construction
GGUF weight loading , computation graph building , KV cache | llama-model.cpp , llama-graph.cpp

GGML Tensor Library & Backend API
Tensor ops , computation graphs , backend dispatch , quantization formats
ggml/include/ggml.h , ggml-backend.h | ggml/src/ggml.c , ggml-backend.cpp

Vulkan Backend
ggml/src/ggml-vulkan/

Instance & Device Management
VkInstance , VkPhysicalDevice , feature detection

Memory & Buffer Management
VkDeviceMemory , VkBuffer , descriptor pools & sets

Pipeline Management
SPIR-V shader modules , VkComputePipeline , caching

Command Buffer & Dispatch
vkCmdDispatch , queue submission , synchronization

GLSL Compute Shaders
matmul softmax rope attention dequant_*
add mul conv2d sum mean argmax ...

Cross-platform GPU: NVIDIA + AMD + Intel

CUDA
ggml-cuda/

CUDA C++ kernels
100+ .cu files
cuBLAS integration

NVIDIA only

Metal
ggml-metal/

Metal Shading Lang
Monolithic .metal
MPS integration

Apple only

CPU
ggml-cpu/

SIMD optimized
AVX / NEON / SVE
Fallback impl

All platforms

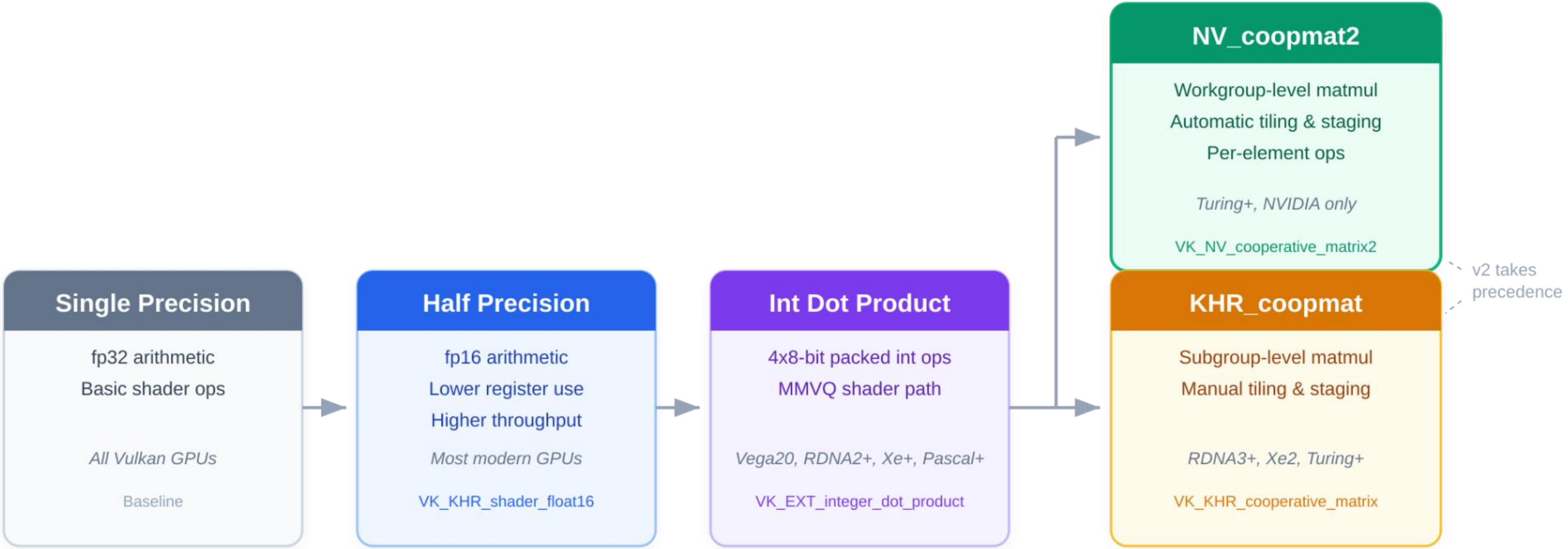
Others
ggml-sycl/ , ggml-hip/

SYCL (Intel)
HIP (AMD ROCm)
OpenCL

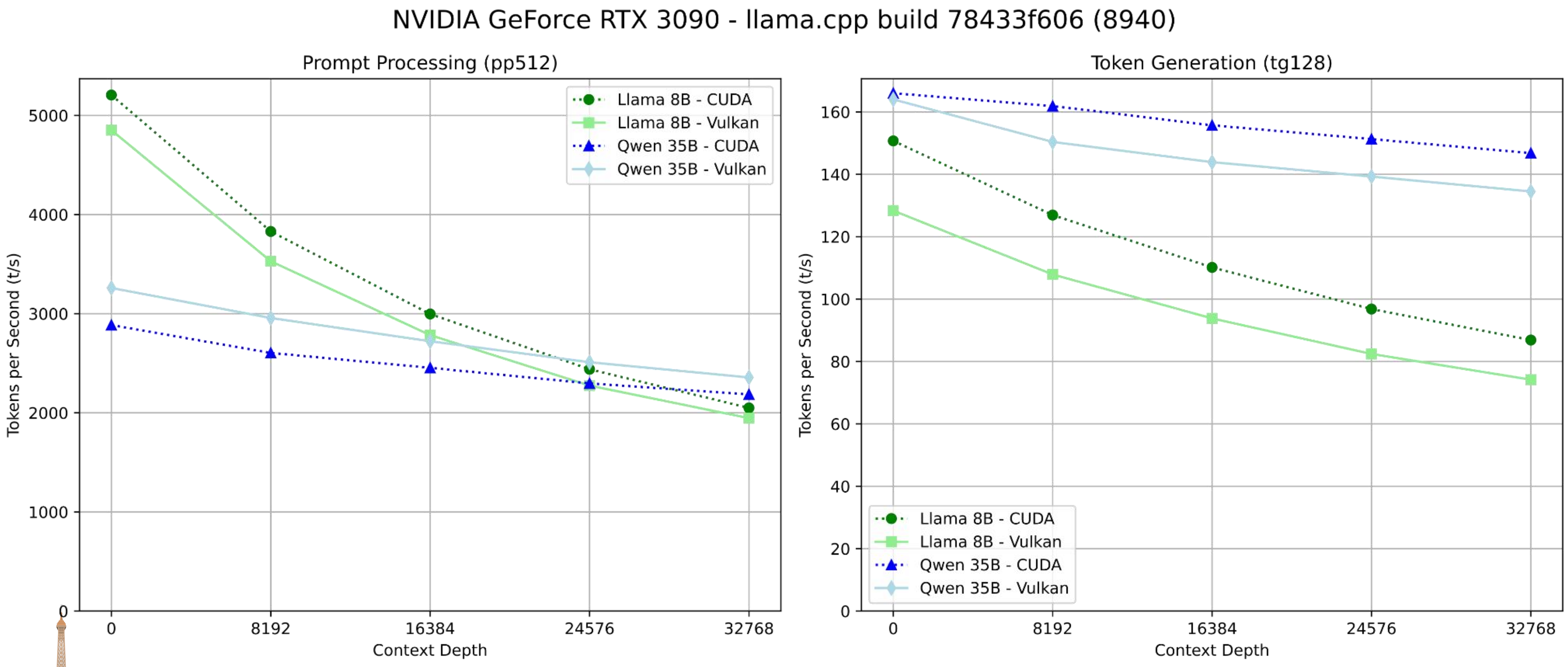
Vendor-specific

All backends implement the same ggml_backend_t vtable interface

The Vulkan Backend

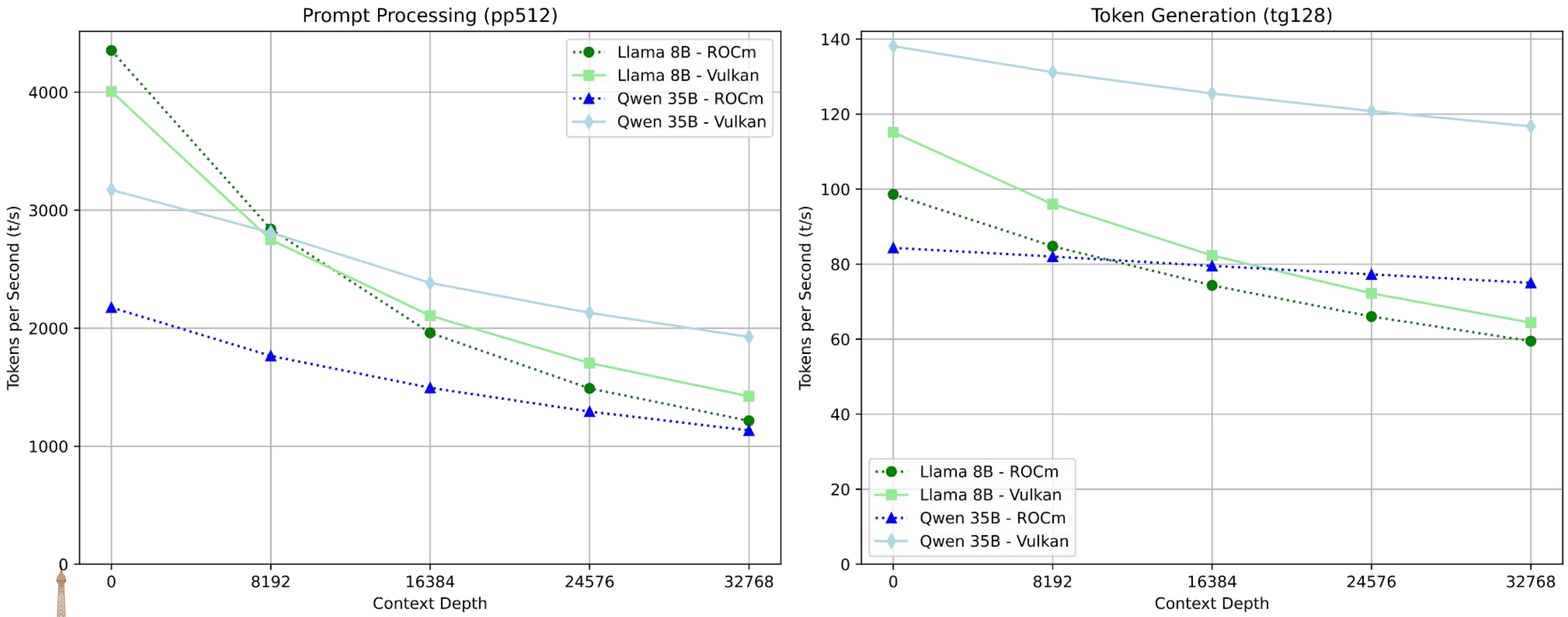


Benchmarks: Nvidia



Benchmarks: AMD

AMD Radeon RX 9070 XT - llama.cpp build 78433f606 (8940)



Driver Pains

```
if (!OLD_AMD_WINDOWS) {
    Of[r][d] += subgroupShuffleXor(Of[r][d], s);
} else {
    // Something about f16vec4 subgroupShuffleXor is broken on AMD Windows RDNA2 and below.
    // Shuffle full vec4 as workaround.
    // See https://github.com/ggml-org/llama.cpp/issues/19881#issuecomment-3958643697
    Of[r][d] += FLOAT_TYPEV4(subgroupShuffleXor(vec4(Of[r][d]), s));
}
```

Driver Pains

```
if (subgroup_size_control_props.maxSubgroupSize == 64 && subgroup_size_control_props.minSubgroupSize == 64) {
    return vk_device_architecture::AMD_GCN;
}
if (subgroup_size_control_props.maxSubgroupSize == 64 && subgroup_size_control_props.minSubgroupSize == 32) {
    // RDNA
    if (shader_core_props_amd.wavefrontsPerSimd == 20) {
        return vk_device_architecture::AMD_RDNA1;
    }
    if (integer_dot_props.integerDotProduct4x8BitPackedMixedSignednessAccelerated) {
        return vk_device_architecture::AMD_RDNA3;
    }
    return vk_device_architecture::AMD_RDNA2;
}
```

```
case VK_VENDOR_ID_AMD:
    if (driver_props.driverID == vk::DriverId::eAmdProprietary || driver_props.driverID == vk::DriverId::eAmdOpenSource) {
        // Workaround for AMD proprietary driver reporting support on all GPUs
        return arch == vk_device_architecture::AMD_RDNA3;
    }
    return true;
```

Driver Pains

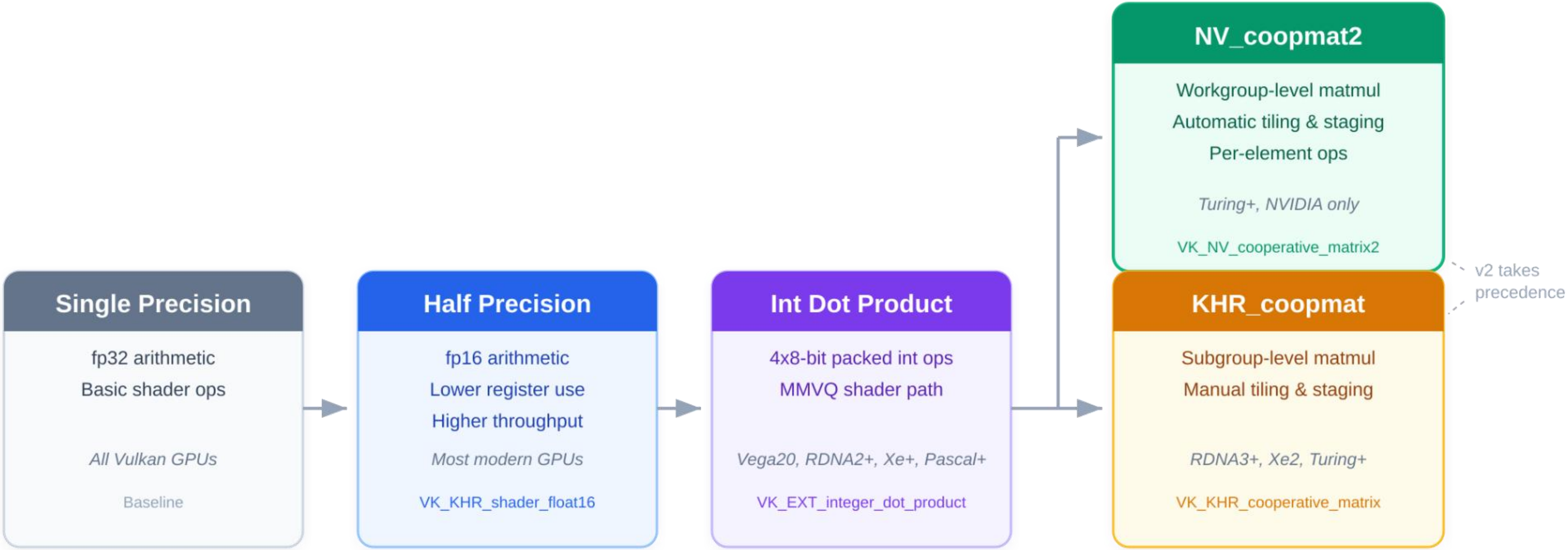

```
if (device->vendor_id == VK_VENDOR_ID_INTEL) {
    // Disable subgroup use due to performance issues when enforcing subgroup sizes
    result.subgroup_size = 32;
    result.disable_subgroups = true;
```

```
if (path == FA_COOPMAT1 && device->architecture == vk_device_architecture::NVIDIA_TURING) {
    // Nvidia compiler bug, see https://github.com/ggml-org/llama.cpp/pull/19075#issuecomment-3820716090
    path = FA_SCALAR;
}
```

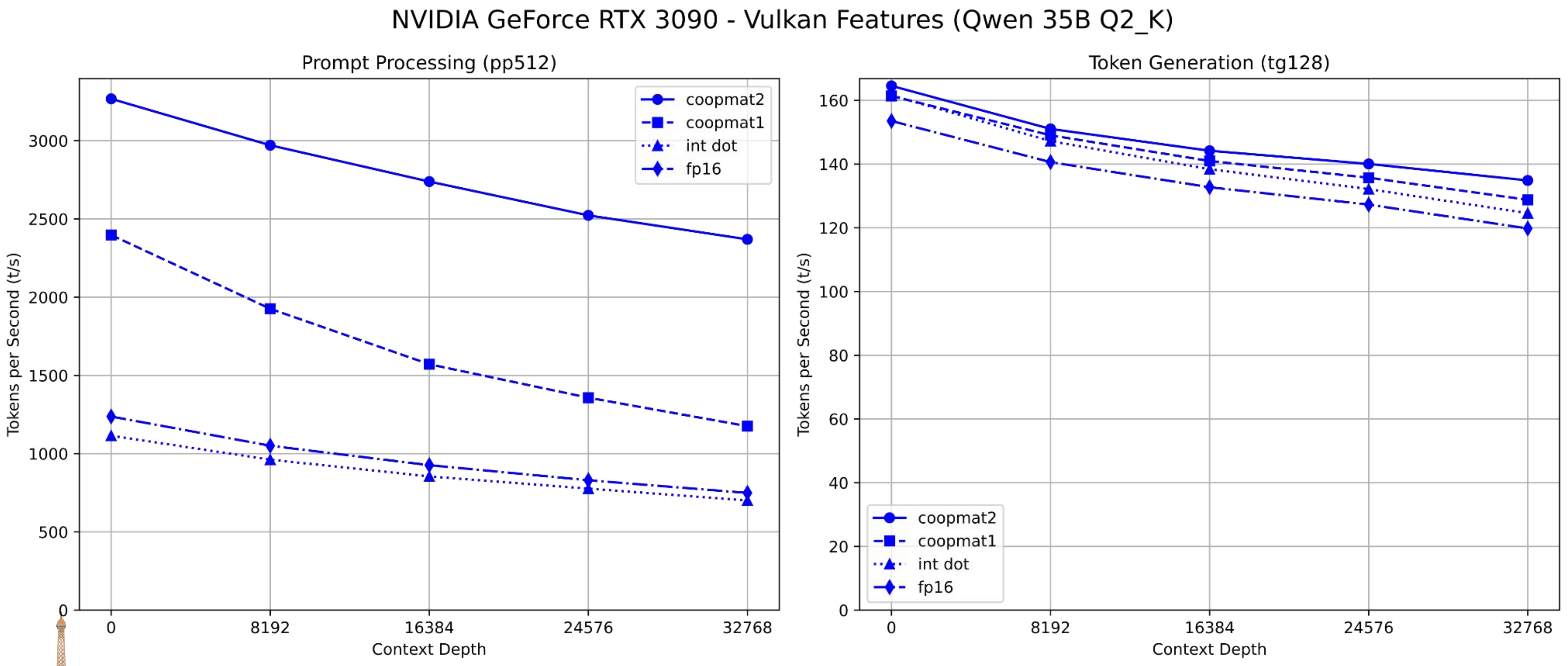
03

The Future

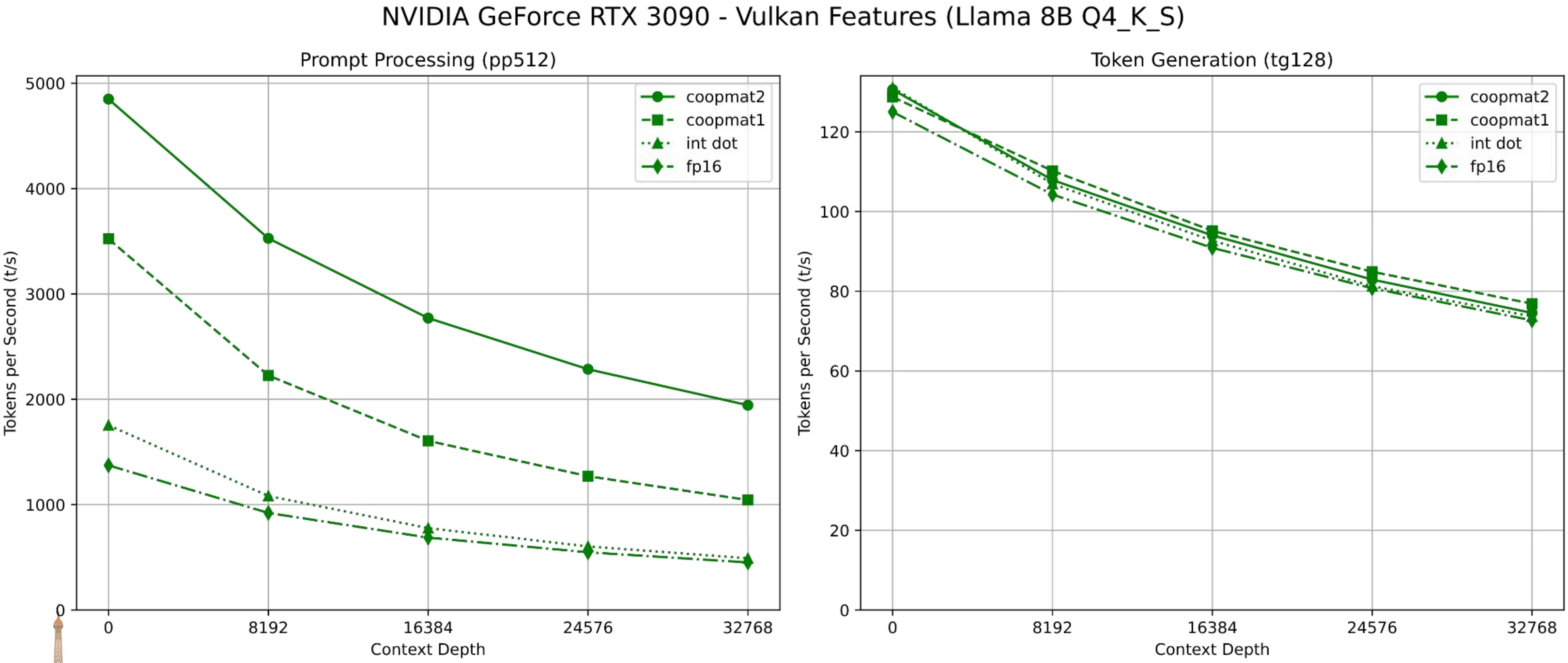
Coopmat in Practice



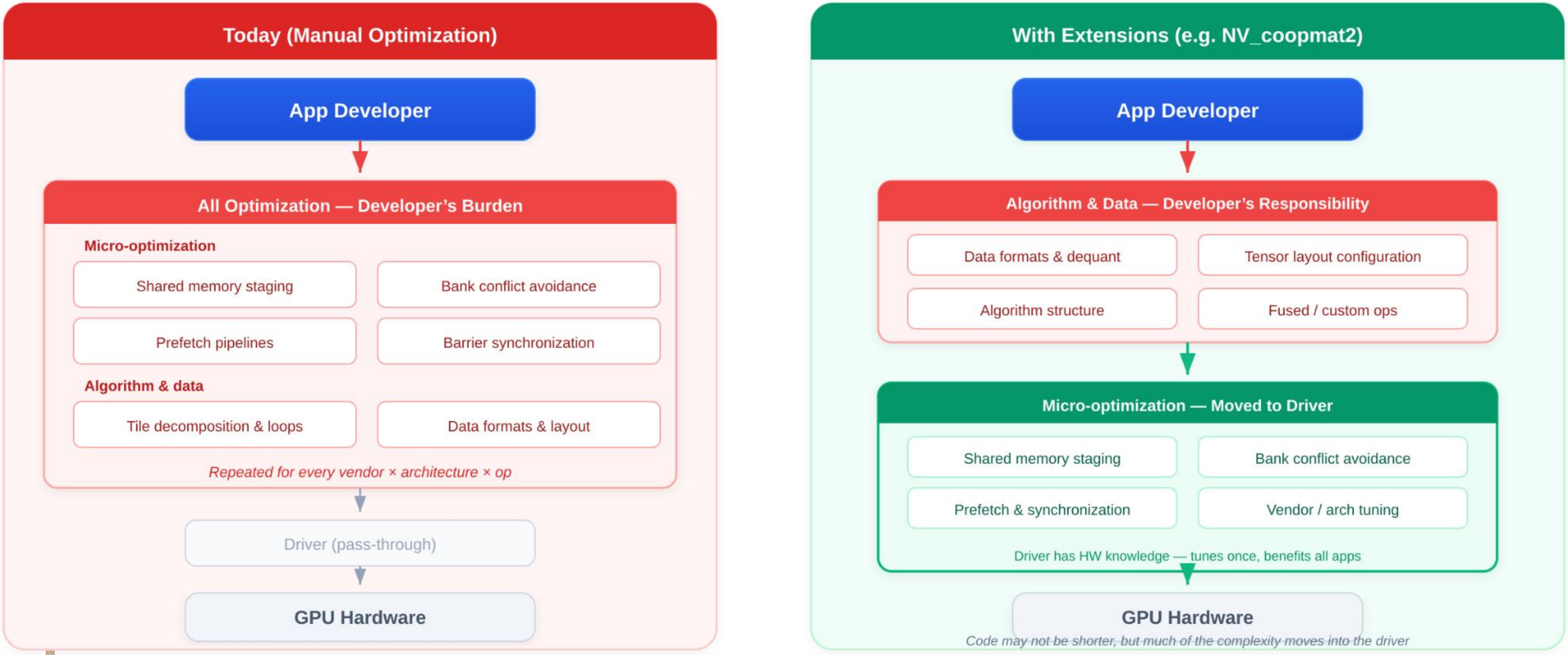
Coopmat in Practice



Coopmat in Practice



The Future


Today (Manual Optimization)
App Developer
All Optimization — Developer's Burden
Micro-optimization
Shared memory staging
Bank conflict avoidance
Prefetch pipelines
Barrier synchronization
Algorithm & data
Tile decomposition & loops
Data formats & layout
Repeated for every vendor × architecture × op
Driver (pass-through)
GPU Hardware
With Extensions (e.g. NV_coopmat2)
App Developer
Algorithm & Data — Developer's Responsibility
Data formats & dequant
Tensor layout configuration
Algorithm structure
Fused / custom ops
Micro-optimization — Moved to Driver
Shared memory staging
Bank conflict avoidance
Prefetch & synchronization
Vendor / arch tuning
Driver has HW knowledge — tunes once, benefits all apps
GPU Hardware
Code may not be shorter, but much of the complexity moves into the driver

GOSIM

GOSIM Paris 2026

04

Conclusion

Conclusion

- Vulkan is a serious option for Edge AI, with great performance and broad compatibility, as demonstrated by GGML.
- It holds real advantages over other Compute APIs (and some disadvantages)
- The optimization burden can be reduced with drivers taking on more work
- It is being actively improved for Machine Learning

GOSIM

GOSIM Paris 2026

Thanks!